Table of Contents

Genetiq Digital Health Twin Platform

---

1. Project Overview

Genetiq is a Ghana-focused **Digital Health Twin Platform** that places AI-powered medical triage and lab-result interpretation directly in the hands of patients . The system creates an interactive, personalized digital representation of the patient's health status, visualized through a 3D human body model that highlights the biological system relevant to each diagnosis.

At its core, Genetiq leverages **Google Gemini (via Google AI Studio)** for multimodal lab analysis, real-time symptom triage, personalized health action plans, and translation into four Ghanaian local languages. A Python/FastAPI fallback layer running local Gemma models provides edge-native inference for environments with limited cloud connectivity.

Genetiq was designed to close the gap between rural Ghanaian patients and the clinical resources they need offering a premium, accessible health experience that works on any device, in any language, even offline.

---

2. Problem Statement

Ghana's healthcare system particularly in rural and peri-urban communities faces 3 critical, intertwined challenges:

1. **Clinical Scarcity**: Rural clinics and Community-based Health Planning and Services (CHPS) compounds are chronically understaffed. Patients wait hours for basic consultations or triage that a trained system could handle in seconds.
2. **Diagnostic Literacy Barriers**: Medical lab reports, Rapid Diagnostic Test (RDT) cassettes, and prescriptions are filled with clinical terminology and numerical biomarkers that patients cannot interpret without professional help. Abnormal values go unnoticed; follow-up care is missed.
3. **Language Barriers**: Critical health guidelines, emergency instructions, and lab interpretations are predominantly published in English, excluding large populations whose primary languages are Twi, Ga, Ewe, or Fante.

Together, these challenges mean that patients frequently have incomplete, inaccessible, or incomprehensible information about their own health, leading to delayed treatment, preventable complications, and unnecessary mortality.

---

3. Solution

Genetiq addresses each of these challenges through a layered, resilient architecture:

- **AI-Powered Triage**: Patients describe their symptoms in plain language. Gemini Flash analyzes the input, identifies the suspected condition, assigns an urgency level (Green / Yellow / Red), and recommends appropriate next steps including when to visit a CHPS compound or call emergency services (112).
- **Multimodal Lab Scanner**: Patients photograph their paper lab reports or RDT cassettes. Gemini Vision performs OCR on the image, extracts all biomarkers, classifies each as normal or abnormal, computes a health score, and delivers a plain-language summary replacing the need for a doctor just to understand the report.

- **3D Digital Twin Visualizer**: Every triage or lab result is mapped to a biological body system (e.g., Hematology, Cardiovascular, Pulmonology). The interactive 3D human body model lights up that system with cinematic camera transitions, giving patients a visual understanding of *where* their health concern is located.
- **Ghanaian Language Support**: All AI responses, lab findings, and recommendations can be dynamically translated into **Twi, Ga, Ewe, and Fante**, with an offline dictionary fallback for common clinical phrases.
- **Smart Offline Fallback**: If the AI server is unreachable, a built-in offline simulator with preset Ghanaian health cases (malaria, typhoid, anemia, UTI) keeps the platform fully functional for demonstrations and real-world offline scenarios.
- **Personalized Action Plans**: Based on lab results, health profile, and symptoms, the AI generates structured 3-category follow-up plans covering medical tests/follow-ups, supplements, and lifestyle modifications.

---

4. Key Features

3D Digital Twin

- Interactive 3D human body model with real-time camera zooming and cinematic transitions.
- Dynamic, glowing system highlights mapped to eight biological regions: Respiratory, Digestive, Endocrine, Renal, Urological, Neurological, Cardiovascular, and Musculoskeletal.
- Driven automatically by AI triage output via Redux state no manual user configuration needed.
- Toggle between full-body view and isolated system views.

Health Dashboard

- Personalized welcome header with user health stats and overall wellness score.
- AI-powered health alerts and recommendation cards.
- Weekly activity visualization with step, calorie, and active minute metrics.
- Quick actions for fast access to lab upload, triage, and history.

Ghanaian AI Health Assistant

- Powered by **Google Gemini 3.5 Flash** via Google AI Studio (`@google/genai` SDK).
- Symptom triage returning structured JSON: urgency level, suspected condition, body system, and actionable advice.
- Multimodal lab scanning image → Gemini Vision OCR → analysis → health score + plain-language findings.
- Ghanaian language translation (Twi, Ga, Ewe, Fante) with offline dictionary fallback.
- Local remedy encyclopedia featuring Moringa, Sobolo, Kontomire, ORS, and Neem tea alongside clinical guidance.

- AI connection badge showing live status: `Gemma AI live`, `Gemma AI live (CPU)`, or `Offline mode`.

Risk Assessment Dashboard

- Glassmorphic risk cards highlighting key health concerns and diagnostic cost indicators.
- Action Plans organized into Follow-up Care, Supplements, and Lifestyle tabs.
- Cardiovascular risk analysis panel.
- Urgency-coded alerts (Green / Yellow / Red).

Health History Tracking

- Paginated history of past lab uploads with smart data charts.
- Side-by-side SVG charts tracking health score trends over time.
- Normal vs. abnormal biomarker visualization.
- Horizontal patient profile banner with quick upload access.

Internationalization (i18n)

- Support for 17 languages including English, Spanish, French, German, Italian, Portuguese, Russian, Arabic, Chinese, Japanese, Korean, Hindi, Polish, Turkish, and the four Ghanaian local languages.
- Implemented using a custom i18n module with locale JSON files.

Authentication & User Management

- Secure registration and login with bcrypt password hashing.
- Session-based user state managed via Redux.
- Avatar image upload and management.
- Real-time online presence tracking via Socket.IO.

Onboarding

- Step-by-step guided health intake (Quiz, Supplements, Uploads, Connections).
- High-fidelity glassmorphic confirmation and success modals.
- Real-time processing visual feedback during Digital Twin generation.

Premium UI/UX

- Dark-first theme with indigo-violet gradient branding.
- Glassmorphism, backdrop blurs, and glow effects for high-risk metrics.
- Framer Motion micro-animations for route transitions and modal state changes.
- Fully responsive across mobile, tablet, and desktop viewports.

---

5. Target Users

User Type	Description
Patients (Rural/Peri-urban Ghana)	Primary audience. Individuals who receive lab reports or RDT results and need plain-language interpretation without needing to see a doctor for basic triage.
Community Health Workers	CHPS compound staff who can use Genetiq as a rapid triage aid during community health visits.
Healthcare Advocates & NGOs	Organizations working to bridge health literacy gaps in West Africa who can deploy Genetiq in low-connectivity field settings.
Medical Students & Researchers	Can use the platform to explore how AI interprets Ghanaian health conditions and local biomarkers.
Software Developers	Developers who want to contribute to or extend the platform the codebase is structured for modularity and clarity.

---

6. Technology Stack

Frontend

Technology	Purpose
React 18.3	Core UI framework
TypeScript 5.x	Static typing and developer experience
Vite 6.x	Build tool and development server
React Three Fiber (R3F)	Declarative Three.js renderer for 3D twin
Three.js r172	3D graphics engine
@react-three/drei	Useful Three.js helpers (cameras, controls, etc.)
@react-three/postprocessing	Post-processing effects (glows, bloom)
Redux Toolkit	Global state management (triage, user, theme)
React Router DOM v7	Client-side routing
Framer Motion	Animations and page transitions
SCSS Modules	Component-scoped styling
Tesseract.js	In-browser OCR for lab image pre-processing
TanStack Query (React Query)	Async data fetching and caching
Socket.IO Client	Real-time messaging
i18next (custom module)	Internationalization
Lucide React / React Icons	Icon library

Backend

Technology	Purpose
Express.js 5.x	Main REST API server

Technology	Purpose
Node.js 18+	Server runtime
Socket.IO	Real-time bi-directional messaging
bcrypt	Password hashing
@supabase/supabase-js	Supabase client for database operations
@google/genai	Official Google Gen AI SDK for Gemini API calls
@google-cloud/storage	Google Cloud Storage for medical file uploads
dotenv	Environment variable management

Database & Authentication

Technology	Purpose
Supabase (PostgreSQL)	Primary relational database (users, messages)
Supabase Auth (planned)	Native authentication layer

AI / ML Services

Technology	Purpose
Google Gemini 3.5 Flash	Primary AI model: lab analysis, symptom triage, action plans, translation (via Google AI Studio)
Gemini Vision (3.5 Flash)	OCR extraction from lab report images
Python 3.10+ / FastAPI	Edge AI gateway for local Gemma model inference
PyTorch + HuggingFace Transformers	Local Gemma model loading and inference
google/gemma-2-2b-it	Default local model (CPU-compatible for development)
google/gemma-4-26b-a4b-it	Production-target model (GPU required)

Cloud & Infrastructure

Technology	Purpose
Google Cloud Storage (GCS)	Encrypted storage for medical lab scans and reports
Google Cloud Run	Serverless containerized deployment (production path)
Vercel	Frontend hosting and deployment
Render	Backend Express server hosting
Docker / docker-compose	Containerization for local and production deployment

Dev Tooling

Technology	Purpose
ESLint + Prettier	Code linting and formatting
Husky + lint-staged	Pre-commit hooks
Vitest + Testing Library	Unit and component testing

Technology	Purpose
concurrently	Run multiple npm scripts in parallel

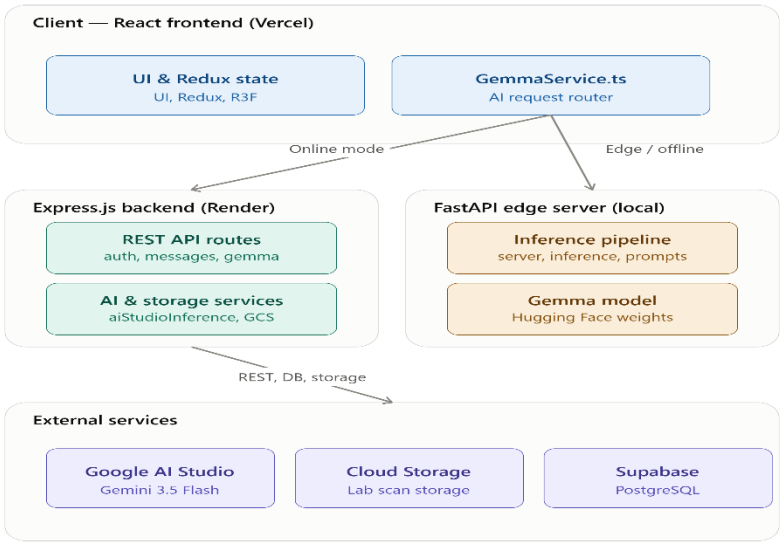
7. System Architecture

Genetiq follows a **three-tier monorepo architecture** with a clear separation between the React frontend, the Node.js/Express API backend, and the Python AI inference layer.

High-Level Flow

1. The React frontend communicates with the Express backend over REST and Socket.IO.
2. The Express backend handles authentication, messaging, and AI orchestration. For AI requests, it calls **Google AI Studio** (Gemini) via the `@google/genai` SDK.
3. For edge-native / offline deployments, the frontend can also route AI requests to a local **FastAPI Python server** that runs Gemma models via Hugging Face Transformers.
4. Lab report images are uploaded to **Google Cloud Storage** from the backend.
5. User data is persisted in **Supabase (PostgreSQL)**.

Architecture Diagram



Component Responsibilities

Component	Responsibility
React Frontend	UI rendering, user interaction, 3D visualization, AI result display, i18n

Component	Responsibility
GemmaService.ts	AI request routing (Gemini cloud vs. Gemma edge vs. offline simulator), health polling
Redux Store	Global state for triage results, body system selection, user session, theme
Express Backend	REST API, CORS, session management, AI orchestration, file upload
aiStudioInference.js	Wraps `@google/genai` SDK sends chat, analyze, action-plan, and translate requests to Gemini
googleCloudService.js	Uploads lab scans to Google Cloud Storage, checks Cloud Run health
FastAPI (Python)	Edge AI server loads Gemma model locally, exposes identical REST endpoints for offline operation
Supabase (PostgreSQL)	Persists user accounts and real-time messages
Google AI Studio	Hosts Gemini Flash for cloud-mode AI inference
Google Cloud Storage	Stores encrypted medical lab images

---

8. Project Structure

```
Genetiq-beta/                          # Monorepo root
├── client/                            #  FRONTEND APPLICATION
│   ├── index.html                     # HTML entry point
│   ├── vite.config.ts                 # Vite bundler + dev proxy config
│   ├── tsconfig.app.json              # TypeScript config for client/src
│   ├── vitest.config.ts               # Unit test config
│   ├── docker-compose.yml             # Local Docker setup
│   ├── Dockerfile                     # Frontend container build
│   ├── public/
│   │   └── assets/
│   │       └── models/
│   │           ├── cardio/            # Cardiovascular 3D model files (.glb)
│   │           └── normal/            # Full-body 3D model files (.glb)
│   └── src/
│       ├── App/                       # App-level configuration
│       │   ├── App.tsx                # Root React component
│       │   ├── main.tsx               # React DOM entry point
│       │   ├── i18n/                  # Internationalization setup
│       │   ├── Redux/                 # Redux store, slices (triageSlice, etc.)
│       │   ├── Routes/                # React Router route definitions
│       │   ├── Styles/                # Global SCSS styles and tokens
│       │   ├── theme/                 # Theme context (dark/light)
│       │   ├── Layouts/               # Shared page layout components
│       │   ├── Providers/             # React context providers
│       │   ├── Services/              # GemmaService.ts — AI API client
```

```
│       │   ├── Hooks/            # Custom hooks (useGemmaConnection, etc.)
│       │   ├── Types/            # Shared TypeScript type definitions
│       │   ├── Consts/           # App-wide constants
│       │   └── Data/             # Static data files (preset cases, etc.)
│       ├── assets/               # Images, SVGs, icons
│       ├── Features/             # Feature-scoped components (not full page
s)
│       │   ├── Auth/             # Login/Register form components
│       │   ├── Dashboard/        # Dashboard widgets (Triage, Health Score,
Activity)
│       │   ├── DigitalTwin/      # 3D body model components (R3F scene)
│       │   ├── Onboarding/       # Onboarding step components
│       │   ├── Risk/             # Risk assessment cards and panels
│       │   └── Structural/       # Shared layout/structural components
│       ├── locales/              # i18n JSON translation files (17 language
s)
│       └── Views/                # Full page-level view components
│           ├── Auth/             # Login and Register pages
│           ├── Dashboard/        # Main dashboard view + AI Assistant
│           ├── DigitalTwin/      # 3D Twin full-screen view
│           ├── HealthHistory/    # Lab history and chart page
│           ├── Landing/          # Marketing/landing page
│           ├── SystemOverview/   # Biological system overview
│           ├── UploadMethod/     # Lab upload and scanner view
│           └── Widgets/          # Standalone widget views
│
├── server/                       #  BACKEND API (Express.js)
│   ├── index.js                  # Express server entry point + Socket.IO s
etup
│   ├── config/
│   │   └── supabase.js           # Supabase client initialization
│   ├── controllers/
│   │   ├── userController.js     # Auth logic (register, login, avatar, log
out)
│   │   └── messageController.js  # Message send/receive logic
│   ├── models/
│   │   ├── userQueries.js        # Supabase queries for users table
│   │   └── messageQueries.js     # Supabase queries for messages table
│   ├── routes/
│   │   ├── auth.js               # /api/auth routes
│   │   ├── messages.js           # /api/messages routes
│   │   └── gemma.js              # /api/gemma routes (AI endpoints)
│   ├── services/
│   │   ├── aiStudioInference.js  # Google AI Studio (@google/genai) wrapper
│   │   ├── googleCloudService.js # GCS upload + Cloud Run health check
│   │   └── prompts.js            # Ghana-tuned system prompts + translation
s
│   ├── gemma/                    #  EDGE AI SERVER (Python/FastAPI)
│   │   ├── server.py             # FastAPI app + all /api/gemma endpoints
│   │   ├── inference.py          # Hugging Face model.generate() helpers
```

```
│   │   ├── prompts.py              # Ghana-specific Python system prompts
│   │   └── requirements.txt        # Python dependencies
│   └── migrations/
│       └── 001_create_initial_schema.sql  # Supabase schema (users + message
s)
│
├── cloudbuild.yaml                 # Google Cloud Build CI config
├── render.yaml                     # Render.com deployment config
├── vercel.json                     # Vercel frontend deployment config
├── package.json                    # Root scripts (dev server orchestration)
├── GEMMA.md                        # Gemma AI implementation + pitch guide
├── SUPABASE_MIGRATION.md           # MongoDB → Supabase migration notes
└── README.md                       # Project overview
```

---

9. Installation Guide

Prerequisites

Ensure the following are installed on your machine:

- **Node.js** v18 or higher
- **npm** v9 or higher
- **Python** 3.10+ (only required for the local Gemma edge AI server)
- **Git**

Step 1 — Clone the Repository

```
git clone https://github.com/agudu50/Genetiq.git
cd Genetiq
```

Step 2 — Install Root Dependencies

```
# From the monorepo root
npm install
```

Step 3 — Install Client Dependencies

```
cd client
npm install
cd ..
```

Step 4 — Install Server Dependencies

```
cd server
npm install
cd ..
```

Step 5 — Configure Environment Variables

Client environment (`client/.env`)

```
cp client/.env.example client/.env
```

Edit `client/.env`:

```
# For local development, point to your local server:
VITE_API_URL=http://localhost:3000
VITE_GEMMA_URL=http://localhost:3000

# For production, use your deployed server URL:
# VITE_API_URL=https://genetiq-server.onrender.com
# VITE_GEMMA_URL=https://genetiq-server.onrender.com
```

Server environment (`server/.env`)

```
cp server/.env.example server/.env
```

Edit `server/.env` with your credentials:

```
# Supabase — Required
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_ANON_KEY=your-anon-key-here

# Server
PORT=3000
NODE_ENV=development
CLIENT_URL=http://localhost:5174

# Google AI Studio — Required for AI features
GEMINI_API_KEY=your-gemini-api-key-here
GEMINI_MODEL=gemini-3.5-flash
GEMINI_VISION_MODEL=gemini-3.5-flash

# Google Cloud — Optional (for lab image storage)
GCP_PROJECT_ID=your-gcp-project-id
GCS_BUCKET_NAME=genetiq-medical-records

# Hugging Face — Required for local Gemma edge server
HF_TOKEN=your-hf-read-token-here
GEMMA_MODEL=models/gemma-4-26b-a4b-it
```

Step 6 — Configure the Database (Supabase)

1. Create a free project at supabase.com.
2. From your Supabase project dashboard, navigate to **SQL Editor**.
3. Create a new query, paste the contents of `server/migrations/001_create_initial_schema.sql`, and run it.

This will create the `users` and `messages` tables with the required indexes.

Step 7 — Run the Application

Option A: Frontend + Express API (recommended for development)

```
# From the client/ directory
cd client
npm run dev
```

This uses `concurrently` to start both the Vite frontend and the Node.js backend simultaneously.

Service	URL
Frontend (Vite)	http://localhost:5174
Backend (Express)	http://localhost:3000

Option B: Frontend + Express API + Local Gemma AI

```
# From the client/ directory
cd client
npm run dev:ai
```

This starts the Vite frontend, the Express backend, **and** the Python FastAPI Gemma server together.

> **Note:** The Gemma edge server requires Python dependencies. Install them first:

```
cd server/gemma
pip install -r requirements.txt
```

Option C: Run Only the Local Gemma AI Server

```
cd client
npm run gemma
```

Step 8 — (Optional) Set Up the Local Gemma Edge AI Server

If you want to use the local edge AI server with actual Gemma model inference:

1. Accept the model license on Hugging Face: google/gemma-2-2b-it
2. Generate a **Read** token at huggingface.co/settings/tokens

3. Start the server:

   PowerShell:

   ```
   $env:HF_TOKEN="your_token_here"
   python server/gemma/server.py --model google/gemma-2-2b-it
   ```

   Command Prompt:

   ```
   set HF_TOKEN=your_token_here
   python server/gemma/server.py --model google/gemma-2-2b-it
   ```

Tip: If the Gemma server is not running, the application automatically enters **Smart Offline Fallback Mode** — all preset Ghanaian medical cases remain testable.

Available Scripts (from `client/` directory)

Command	Description
`npm run dev`	Start frontend (Vite) + backend (Express) concurrently
`npm run dev:ai`	Start frontend + backend + Gemma Python server
`npm run gemma`	Start only the Gemma Python server
`npm run build`	TypeScript compile + Vite production build
`npm run preview`	Preview the production build locally
`npm run lint`	Run ESLint
`npm run format`	Format all files with Prettier

Docker Deployment

```
# Build and start all services with Docker Compose
docker-compose up --build

# Or build and run the container manually
docker build -t genetiq-app .
docker run -p 3000:3000 genetiq-app
```

---

10. Environment Variables

Client (`client/.env`)

Variable	Description	Example
`VITE_API_URL`	Backend Express server URL	`http://localhost:3000`
`VITE_GEMMA_URL`	AI API server URL (Express or Python FastAPI)	`http://localhost:3000`

Server (`server/.env`)

Variable	Description	Required	Example
`SUPABASE_URL`	Your Supabase project URL	Yes	`https://xyz.supabase.co`
`SUPABASE_ANON_KEY`	Supabase anonymous/public key	Yes	`eyJhbGci...`
`PORT`	Port the Express server listens on	No	`3000`
`NODE_ENV`	Runtime environment	No	`development`
`CLIENT_URL`	Frontend origin for CORS	No	`http://localhost:5174`
`GEMINI_API_KEY`	Google AI Studio API key	Yes (for AI features)	`AIza...`
`GEMINI_MODEL`	Gemini model ID for chat/analysis	No	`gemini-3.5-flash`
`GEMINI_VISION_MODEL`	Gemini model ID for image OCR	No	`gemini-3.5-flash`
`GEMMA_MODEL`	Gemma model ID (via AI Studio or HF)	No	`models/gemma-4-26b-a4b-it`
`GCP_PROJECT_ID`	Google Cloud project ID	No	`my-gcp-project`
`GCS_BUCKET_NAME`	GCS bucket for lab report uploads	No	`genetiq-medical-records`
`HF_TOKEN`	Hugging Face read token (for gated models)	For local Gemma	`hf_...`
`HF_S3_ACCESS_KEY_ID`	Hugging Face S3 access key (optional)	No	—
`HF_S3_SECRET_ACCESS_KEY`	Hugging Face S3 secret (optional)	No	—

Never commit real API keys or secrets to version control. All `.env` files are listed in `.gitignore`.

---

11. Usage Guide

Registering & Logging In

1. Navigate to the landing page and click **Get Started**.
2. On the **Register** page, enter a username, email, and password.
3. Log in with your username and password.
4. On first login, you will be prompted to set a profile avatar.

Running a Symptom Triage

1. From the **Dashboard**, locate the **AI Health Assistant / Triage Widget**.
2. Type your symptoms in plain English (e.g., *"I have a fever, chills, and a headache for 3 days"*).
3. The AI will respond with:
   - A plain-language assessment of your symptoms.
   - A suspected condition (e.g., *Malaria*).
   - An urgency level: **Green** (monitor at home), Yellow (visit CHPS soon), **Red** (seek emergency care immediately).
   - The **3D Digital Twin** will automatically highlight the relevant body system.
4. Optionally, toggle the language dropdown to receive the response in **Twi, Ga, Ewe, or Fante**.

Uploading a Lab Report

1. Navigate to **Upload Lab Report** (via Quick Actions or the Upload view).
2. Upload a photo or scan of your paper lab report or RDT cassette.
3. Alternatively, select a **Preset Case** (Malaria, Typhoid, Anemia, etc.) for a demonstration.
4. The AI will:
   - Extract all biomarkers via OCR.
   - Classify each as Normal, Borderline, or Abnormal.
   - Compute an overall **Health Score** (0–100).
   - Generate a plain-language summary with ranked findings.
5. A **Generate Action Plan** option becomes available after analysis.

Viewing Your Action Plan

1. After a lab analysis, click **Generate Action Plan**.
2. The AI creates a 3-category personalized plan:
   - **Follow-up Care**: Recommended medical tests and checkup schedule.
   - **Supplements**: Evidence-based supplement recommendations.
   - **Lifestyle**: Daily habits, diet changes, and activity adjustments.

3. Navigate between tabs using the pill-shaped category selectors.

Exploring the 3D Digital Twin

1. Navigate to the **Digital Twin** view.
2. Interact with the 3D body model:
   - **Click** on a body system to highlight it.
   - The camera automatically transitions with a cinematic zoom.
   - Systems illuminate with colored glows indicating your health status.
3. Triage results automatically drive the twin run a triage and watch the model update in real time.

Viewing Health History

1. Navigate to **Health History**.
2. Browse paginated cards of past lab uploads.
3. Select a record to view its detailed chart breakdown health score trends and normal-vs-abnormal marker comparison over time.

---

12. API Documentation

The Express.js backend exposes three route groups. All endpoints are prefixed with their respective path and served on the configured `PORT` (default: `3000`).

Authentication API (`/api/auth`)

`POST /api/auth/register`

Register a new user account.

- **Auth required**: No
- **Request body**:

```
{
  "username": "john_doe",
  "email": "john@example.com",
  "password": "securepassword123"
}
```

- **Success response** `200`:

```
{
  "status": true,
  "user": {
```

```
    "id": "uuid-v4",
    "username": "john_doe",
    "email": "john@example.com",
    "isAvatarImageSet": false,
    "avatarImage": ""
  }
}
```

- **Failure response** `200`:

```
{ "status": false, "msg": "Username already used" }
```

`POST /api/auth/login`

Authenticate an existing user.

- **Auth required**: No
- **Request body**:

```
{ "username": "john_doe", "password": "securepassword123" }
```

- **Success response** `200`:

```
{
  "status": true,
  "user": { "id": "uuid", "username": "john_doe", "email": "..." }
}
```

- **Failure response** `200`:

```
{ "status": false, "msg": "Incorrect Username or Password" }
```

`GET /api/auth/allusers/:id`

Get all registered users except the specified user (used for messaging).

- **Auth required**: No (session-managed client-side)
- **URL params**: `id` UUID of the current user
- **Success response** `200`: Array of user objects `[{ id, username, email, avatarImage }]`

`POST /api/auth/setavatar/:id`

Set or update a user's avatar image.

- **Request body**: `{ "image": "<base64-string>" }`

- **Success response** `200`: `{ "isSet": true, "image": "<base64>" }`

GET /api/auth/logout/:id

Remove a user from the online users map (Socket.IO cleanup).

- **Success response** `200`: Empty body

Messages API (`/api/messages`)

POST /api/messages/addmsg/

Send a new message between two users.

- **Request body**: `{ "from": "sender-uuid", "to": "recipient-uuid", "message": "Hello!" }`
- **Success response** `200`: `{ "msg": "Message added successfully.", "status": true }`

POST /api/messages/getmsg/

Retrieve the message history between two users.

- **Request body**: `{ "from": "sender-uuid", "to": "recipient-uuid" }`
- **Success response** `200`: Array of `{ fromSelf: boolean, message: string }` objects

AI / Gemma API (`/api/gemma`)

GET /api/gemma/health

Check AI service readiness and configuration.

- **Auth required**: No
- **Success response** `200`:


```
{
  "status": "ok",
  "model_loaded": true,
  "model_id": "models/gemini-3.5-flash",
  "sdk": "@google/genai",
  "device": "google-ai-studio",
  "supports_vision": true,
  "google_cloud": {
    "platform": "Local / Custom Host",
    "cloud_run": false,
```

```
    "gcs_configured": false,
    "project_id": "not-configured"
  }
}
```

POST /api/gemma/analyze

Analyze a lab report image or text. Accepts a photo (base64), extracted text, or a preset case ID.

- **Auth required**: No
- **Request body**:

```
{
  "image_base64": "<base64-encoded-image-or-data-url>",
  "lab_text": "Optional: pre-extracted text from OCR",
  "preset_id": "malaria_anemia",
  "patient_age": "35",
  "patient_gender": "male",
  "language": "english"
}
```

- **Success response** `200`:

```
{
  "health_score": 62,
  "summary": "Your haemoglobin is low, which may indicate mild anaemia.
..",
  "findings": [
    {
      "marker": "Haemoglobin",
      "value": "9.2 g/dL",
      "status": "Low",
      "statusLabel": "Abnormal",
      "note": "Below normal range; may indicate anaemia."
    }
  ],
  "recommendations": [
    {
      "title": "See a Doctor",
      "body": "Visit your nearest CHPS compound for follow-up malaria t
esting."
    }
  ]
}
```

- **Error response** `400`: `{ "detail": "Provide preset_id, lab_text, or image_base64" }`

POST /api/gemma/chat

Symptom triage via natural language conversation.

- **Auth required**: No
- **Request body**:

```
{
  "message": "I have had a fever and headache for two days.",
  "language": "english",
  "image_base64": "<optional-image>"
}
```

- **Success response** `200`:

```
{
  "message": "Your symptoms are consistent with malaria. Seek care at y
our local CHPS compound today...",
  "bodySystem": "Hematology",
  "urgency": "Yellow",
  "condition": "Suspected Malaria",
  "system": "Hematology"
}
```

POST /api/gemma/action-plan

Generate a personalized 3-category health action plan based on lab results and patient profile.

- **Auth required**: No
- **Request body**:

```
{
  "patient_age": "35",
  "patient_gender": "male",
  "health_score": 62,
  "summary": "Anaemia indicators present...",
  "findings": [...],
  "recommendations": [...],
  "symptoms": ["fatigue", "headache"],
  "medical_conditions": ["hypertension"],
  "medications": [{ "name": "Amlodipine", "dosage": "5mg", "frequency":
"daily" }],
  "lifestyle": { "smoking": "no", "alcohol": "occasional", "exercise":
"low", "diet": "mixed" },
```

```
  "language": "english"
}
```

- **Success response** `200`:

```
{
  "follow_up_care": [{ "title": "...", "body": "..." }],
  "supplements": [{ "title": "Iron-rich foods", "body": "Eat Kontomire,
beans, and dark leafy greens daily..." }],
  "lifestyle": [{ "title": "Hydration", "body": "Drink at least 2 litre
s of water daily..." }],
  "source": "gemma"
}
```

`POST /api/gemma/translate`

Translate health text into a Ghanaian local language.

- **Request body**: `{ "text": "Your haemoglobin is low.", "language": "twi" }`
- **Success response** `200`:

  ```
  { "translation": "Wo hemoglobin yɛ ketewa.", "source": "gemma" }
  ```

  (Source can be `"offline"` if the phrase is in the local dictionary, or `"gemma"` for full AI translation.)

---

13. Database Design

Genetiq uses **Supabase (PostgreSQL)** for persistent data storage. The current schema is minimal, covering user accounts and messages. Health and lab data is computed client-side and returned by the AI service without being persisted to the database.

Tables

`users`

Column	Type	Constraints	Description
`id`	UUID	PRIMARY KEY, DEFAULT `gen_random_uuid()`	Unique user identifier
`username`	VARCHAR(20)	UNIQUE, NOT NULL	User's display name
`email`	VARCHAR(50)	UNIQUE, NOT NULL	User's email address
`password`	VARCHAR(255)	NOT NULL	bcrypt-hashed password

Column	Type	Constraints	Description
`isAvatarImageSet`	BOOLEAN	DEFAULT `false`	Whether the user has set an avatar
`avatarImage`	TEXT	DEFAULT `''`	Base64-encoded avatar image
`created_at`	TIMESTAMP	DEFAULT `NOW()`	Account creation timestamp
`updated_at`	TIMESTAMP	DEFAULT `NOW()`	Last update timestamp

messages

Column	Type	Constraints	Description
`id`	UUID	PRIMARY KEY, DEFAULT `gen_random_uuid()`	Unique message identifier
`message`	TEXT	NOT NULL	The message text content
`users`	UUID[]	NOT NULL	Array of both participant UUIDs
`sender`	UUID	REFERENCES `users(id)` ON DELETE CASCADE	UUID of the sending user
`created_at`	TIMESTAMP	DEFAULT `NOW()`	Message send timestamp
`updated_at`	TIMESTAMP	DEFAULT `NOW()`	Last update timestamp

Entity Relationship Diagram

```
erDiagram
    USERS {
        UUID id PK
        VARCHAR username
        VARCHAR email
        VARCHAR password
        BOOLEAN isAvatarImageSet
        TEXT avatarImage
        TIMESTAMP created_at
        TIMESTAMP updated_at
    }

    MESSAGES {
        UUID id PK
        TEXT message
        UUID[] users
        UUID sender FK
        TIMESTAMP created_at
        TIMESTAMP updated_at
    }
```

```
USERS ||--o{ MESSAGES : "sends"
```

Indexes

Index	Table	Column	Purpose
`idx_users_username`	`users`	`username`	Fast login lookup
`idx_users_email`	`users`	`email`	Fast email uniqueness check
`idx_messages_sender`	`messages`	`sender`	Fast message history retrieval

---

14. Authentication and Security

Authentication Method

Genetiq currently uses **custom username/password authentication** backed by Supabase PostgreSQL, managed via the Express.js API.

- **Registration**: Passwords are hashed using `bcrypt` with a salt round of `10` before storage. No plaintext passwords are ever persisted.
- **Login**: On login, bcrypt compares the provided password against the stored hash. The password field is removed (`delete user.password`) from the response object before it is sent to the client.
- **Session Management**: Upon successful login, the user object is stored in the React/Redux client-side state. There is no server-side session or JWT; the frontend manages the authentication state in memory and Redux.
- **Logout**: The user's ID is removed from the in-memory `global.onlineUsers` Map (used by Socket.IO for real-time presence tracking). The client clears its Redux state.

Planned Enhancement: Supabase Auth (JWT-based, with Row Level Security) is documented as a planned upgrade in the migration guide.

Authorization

- Route authorization is currently handled client-side: unauthenticated users are redirected to the Login page by React Router guards.
- API endpoints are not protected by server-side middleware tokens in the current version. This is identified as an area for improvement.

Password Security

- `bcrypt` with salt rounds `10` computationally expensive to brute-force.
- Passwords are never returned in any API response.

Real-Time Security (Socket.IO)

- Socket connections are tracked in a server-side `global.onlineUsers` Map keyed by user ID.
- Direct messages are routed only to the specific socket ID of the recipient.

AI Data Privacy

- **Patient images are never sent to third-party cloud services beyond Google AI Studio** (the user's own configured Gemini API key).
- When Google Cloud Storage is configured, lab images are uploaded under the `lab-scans/` prefix with timestamp-based filenames.
- Every AI prompt includes an explicit disclaimer: *"This is AI-powered health guidance, not a medical diagnosis. Always consult a qualified healthcare provider."*

Input Validation

- Lab text inputs are capped at 4,000 characters before being sent to the AI to prevent prompt injection and excessive token usage.
- Base64 image inputs are sanitized by extracting MIME type using regex before being passed to the AI SDK.
- The Express server accepts a maximum body size of `200MB` (required for base64-encoded medical images).

CORS

- CORS is configured to allow all origins in development (`origin: (origin, callback) => callback(null, true)`).

  For production, the `CLIENT_URL` environment variable should be set and the CORS policy should be restricted to that specific origin.

---

15. Challenges and Solutions

Challenge 1: Edge AI Inference Constraints (CPU vs. GPU vs. Offline)

Problem: Running large language models like Gemma 4 (12B parameters) locally on developer machines requires significant GPU resources that most users do not have. Running on CPU is feasible but extremely slow for real-time chat a single response could take several minutes, making the experience unusable.

Solution: A three-tier resilience strategy was implemented in `GemmaService.ts`:

Tier	Condition	Behaviour
Tier 1 (Cloud)	Gemini API key present	All AI requests go to Google AI Studio (Gemini Flash) fast, multimodal
Tier 2 (CPU Edge)	Local Gemma server running on CPU	Chat uses an instant rule-based triage simulator; lab analysis uses browser-side Tesseract.js OCR piped to a text-only Gemma prompt
Tier 3 (Offline)	Server unreachable	Offline simulator with preset Ghanaian case data (malaria, typhoid, anemia, UTI)

The AI connection status is surfaced live to the user via a badge (`Gemma AI live` / `Offline mode`) so there is never a silent failure.

A health polling hook (`useGemmaConnection`) checks `/api/gemma/health` every 15 seconds and caches the result to avoid redundant network requests.

Challenge 2: Multimodal Lab Scanning Without Exposing Patient Data

Problem: Patients' medical lab report photos contain sensitive personally identifiable information (PII). Sending raw images to a public cloud OCR API without patient consent or proper data handling agreements raises significant privacy and compliance concerns, especially in a healthcare context.

Solution: A two-stage approach was adopted:

1. **Client-side pre-processing with Tesseract.js**: Before any image is sent to the server, the browser runs Tesseract.js OCR locally to extract the text content from the image. In many cases, this extracted text is sufficient for Gemini to perform the analysis without the image ever leaving the device.
2. **Server-side Gemini Vision as fallback**: If client-side OCR yields insufficient text (fewer than 10 characters), the base64-encoded image is sent server-side to Gemini Vision, which performs a more accurate transcription. The transcribed text is then passed to the analysis model rather than the raw image, decoupling the potentially sensitive image from the analytical AI call.
3. **Google Cloud Storage for archiving**: If GCS is configured, lab images are stored in an encrypted GCS bucket under a timestamped, non-identifying path — not in the application database.

This approach minimises the exposure of raw patient images while maintaining high-quality analysis.

Challenge 3: Database Migration (MongoDB → Supabase/PostgreSQL)

Problem: The initial version of Genetiq was built on MongoDB/Mongoose. MongoDB's document model required a full teardown of the data layer when the team decided to migrate to Supabase/PostgreSQL for better structured querying, built-in auth capabilities, and Row Level Security.

Solution: All Mongoose model files were replaced with clean Supabase query modules (`userQueries.js`, `messageQueries.js`) that use the `@supabase/supabase-js` client. The MongoDB `_id` (ObjectId) pattern was replaced with PostgreSQL UUIDs generated by `gen_random_uuid()`. A single SQL migration file (`001_create_initial_schema.sql`) defines the entire schema, making it reproducible in any Supabase project.

Challenge 4: Ghana-Specific AI Calibration

Problem: Generic large language models are trained predominantly on Western medical datasets. Conditions common in Ghana (malaria, typhoid, sickle cell, schistosomiasis), local care pathways (CHPS compounds, emergency lines 112/193), and local food-based remedies (Moringa, Sobolo, Kontomire) are underrepresented or absent in default model behaviour.

Solution: All AI requests are prefixed with carefully crafted **Ghana-tuned system prompts** (`server/services/prompts.js` and `server/gemma/prompts.py`) that explicitly encode:

- A prioritized list of locally endemic diseases and their typical biomarker signatures.
- The Ghanaian healthcare escalation pathway (self-care → CHPS → district hospital → emergency).
- A curated local remedy encyclopedia paired with mandatory "see a doctor" disclaimers.
- Structured JSON output schemas so the UI can render cards, scores, and 3D twin highlights reliably without parsing free text.

---

16. Future Improvements

Feature Roadmap

Priority	Feature	Description
High	**Supabase Auth Migration**	Replace the custom username/password flow with Supabase Auth (JWT, OAuth, MFA) and enforce Row Level Security (RLS) on all tables.
High	**Server-side Route Protection**	Add JWT middleware to all Express API routes to prevent unauthorized access.
Medium	**Wearable Device Integration**	Connect Apple Health, Fitbit, and Oura Ring via their APIs to pull real-time biometric data into the Digital Twin.
Medium	**AI-Powered Predictive Analytics**	Use historical lab data trends to predict health trajectory and flag emerging risks before they become critical.
Medium	**Family Health History Tracking**	Allow users to create and manage linked family member profiles for hereditary condition screening.
Medium	**Telemedicine Integration**	Enable in-app video consultations with registered Ghanaian doctors, with AI-generated triage summaries shared automatically.
Medium	**Gemma 4 GPU Deployment**	Deploy `google/gemma-4-26b-a4b-it` on a dedicated GPU cloud instance (Fly.io, RunPod) to unlock full multimodal on-device inference.

Priority	Feature	Description
Low	**Lab Report PDF Parsing**	Extend the scanner to accept PDF lab reports (using `pdfjs-dist`, already installed) in addition to image uploads.
Low	**Additional Ghanaian Languages**	Expand language support to include Hausa, Dagbani, and Nzema.
Low	**Offline Data Persistence**	Use `localforage` (already installed) to persist recent lab results and action plans for fully offline use.
Low	**Progressive Web App (PWA)**	Add a service worker and Web App Manifest so Genetiq can be installed on mobile devices and used without a browser.
Low	**CHPS Compound Directory**	Integrate a searchable map of CHPS compounds and health facilities by district, powered by Ghana Health Service open data.

---

17. Contributors

Name	Role
Tech Tutors	Core development team

---

18. License

---

19. Conclusion

Genetiq represents a meaningful intersection of frontier AI, interactive 3D visualization, and culturally adapted healthcare built specifically for the Ghanaian context. By combining Google Gemini's multimodal capabilities with a resilient offline fallback architecture, Genetiq ensures that a patient in a rural CHPS compound can receive the same quality of lab interpretation and symptom triage as someone in a well-resourced urban clinic.

The platform's 3D Digital Twin makes abstract medical data tangible and accessible patients *see* where their health issue is, not just read clinical jargon. The four Ghanaian language modes and local remedy encyclopedia ensure that culturally relevant, actionable guidance reaches users in the way they understand best.

Beyond its immediate health impact, Genetiq demonstrates a practical model for deploying edge-native AI in low-resource environments: graceful degradation from cloud to CPU to offline, strict JSON-based AI contracts, and privacy-by-design image handling.

As the platform evolves with Supabase Auth hardening, wearable integrations, and GPU-hosted Gemma 4 inference Genetiq is positioned to grow from a clinical literacy tool into a comprehensive, proactive digital health companion for Ghanaians everywhere.

Made with by the Tech Tutors team